SIMATS Engineering

Department of Information Technology

CSA47 – Deep Learning

Assignment 1 – CO1: Learning Algorithms, Estimation and Neural Network Foundations

Student Name: _____Shaik Fazul_____ **Total marks:** 50

Register No.: 192411063

Scenario: Smart Manufacturing Quality Inspection (defect prediction from temperature, vibration, pressure and humidity sensors)

Task 1 – Learning Algorithms: Task, Experience and Performance

QUESTION (4 Marks)

The factory has 5,000 past inspection records. Each record contains four sensor readings and a label stating whether the component was "defective" or "acceptable". The system must label new components automatically.

a) Define the learning problem using Task (T), Experience (E) and Performance measure (P).

b) Identify the learning paradigm used and justify why a fixed hand-written rule is less suitable than a learning algorithm.

c) Explain why accuracy alone may be misleading if only 3% of components are defective, and name two better measures.

ANSWER

(a) T, E, P definition

Element	Definition
Task (T)	Binary classification: given four sensor readings (temperature, vibration, pressure, humidity) of a component, predict "defective" or "acceptable".
Experience (E)	5,000 historical inspection records, each a 4-feature vector x with a human-assigned label y ∈ {defective, acceptable}.
Performance (P)	Quality of predictions on unseen components – e.g. recall and precision on the defective class (and F1), measured on a held-out test set.

Formally (Mitchell's definition), a program is said to learn from experience E with

respect to a class of tasks T and performance measure P if its performance on T, as measured by P, improves with experience E. Here the inspection system improves its ability to label components as it is shown more labelled records. The input space is $X = \mathbb{R}^4$ (temperature, vibration, pressure, humidity) and the output space is Y = {defective, acceptable}; the learner must find a function $f: X \rightarrow Y$ that works well not only on the 5,000 stored records but also on components that have never been seen (generalization). To measure this honestly, the records are divided into training, validation and test subsets (for example 70/15/15), and P is always reported on the test subset that was not used for fitting or tuning.

(b) Paradigm and why not a fixed rule

This is **supervised learning** (classification) because every training example comes with a ground-truth label. A fixed hand-written rule (e.g. "defective if vibration > 7 and temperature > 90") is less suitable because:

- Defects depend on interactions among sensors and on non-linear boundaries that engineers cannot easily write down.
- Thresholds are brittle: sensor drift, new machines or new materials change the data and the rule must be rewritten by hand, whereas a learning algorithm is simply retrained on new data.
- A learned model outputs a probability and its error can be measured and improved from data; a rule gives no such measure of confidence.

The paradigm is therefore supervised, as opposed to unsupervised learning (no labels, e.g. clustering sensor patterns) or reinforcement learning (learning from rewards through interaction). Because the output is a discrete category rather than a continuous value, the task is classification, not regression. Labels are expensive because a human inspector must examine each part, which is why the 5,000 labelled records are valuable "experience".

(c) Accuracy with 3% defective

A trivial classifier that predicts "acceptable" for every component is 97% accurate, yet it detects **none** of the defective parts – the very events the factory cares about. Accuracy is dominated by the majority class and hides poor minority-class performance. Better measures:

- **Recall (sensitivity) = TP/(TP+FN):** fraction of real defects caught (the trivial classifier scores 0).
- **Precision = TP/(TP+FP)** (or the F1-score, their harmonic mean, or AUC-PR):

fraction of flagged components that are really defective.

Worked example. Out of 5,000 records, 3% = 150 are defective and 4,850 are acceptable. A "lazy" model that always predicts acceptable has TP = 0, FN = 150, FP = 0, TN = 4,850, so accuracy = 4,850/5,000 = 97% but recall = 0/150 = 0%. Now take a useful model with TP = 120, FN = 30, FP = 60, TN = 4,790: accuracy = (120 + 4,790)/5,000 = 98.2% (only 1.2 points better), but recall = 120/150 = 0.80, precision = 120/180 = 0.667 and F1 = 2PR/(P+R) ≈ 0.727. The accuracy figures barely separate the two models, while recall and F1 separate them completely. In inspection, a missed defect (false negative) is usually more costly than a false alarm, so recall is monitored closely while precision keeps the number of unnecessary re-inspections under control.

Task 2 – Capacity, Overfitting and Generalization

QUESTION (3 Marks)

Three candidate models are evaluated. Model X has a training error of 18% and a validation error of 20%. Model Y has a training error of 2% and a validation error of 19%. Model Z has a training error of 6% and a validation error of 8%.

a) Identify which model underfits, which overfits and which generalizes best.

b) Explain how model capacity and the generalization gap are related.

c) Suggest one suitable improvement each for Models X and Y.

ANSWER

(a) Identification

Model	Train err.	Val. err.	Gap	Diagnosis
X	18%	20%	2%	Underfits (high bias – error high on both sets)
Y	2%	19%	17%	Overfits (high variance – large gap)
Z	6%	8%	2%	Generalizes best (lowest validation error, small gap)

(b) Capacity and generalization gap

Capacity is the range of functions a model can fit. With too little capacity (Model X) the model cannot even fit the training data, so training error is high (underfitting). As capacity grows, training error keeps falling, but beyond an optimum the model starts

memorizing noise, so validation error rises and the generalization gap (validation error − training error) widens (Model Y). The expected test error therefore forms a U-shape against capacity, and the best model sits near its minimum (Model Z).

(c) Remedies

- **Model X (underfit):** increase capacity – add hidden neurons/layers or non-linear features, and train longer – so the model can capture the pattern.
- **Model Y (overfit):** add regularization (L2 weight decay, dropout or early stopping) or collect more training data/reduce model size, shrinking the gap.

Learning curves. Plotting training and validation error against model capacity (or training-set size) makes the diagnosis visible. For an underfitting model both curves are high and close together: adding more data does not help, only more capacity does. For an overfitting model the training curve is low, the validation curve is high and the gap closes only slowly as more data is added. A well-fitted model has both curves low and close together. Applying this to the three models: X (18%, 20%) has high bias, Y (2%, 19%) has high variance, and Z (6%, 8%) has an acceptable bias and variance balance.

Bias–variance view. Expected test error = bias² + variance + irreducible noise. Low-capacity models have large bias and small variance; high-capacity models have small bias and large variance. The purpose of model selection is to choose the capacity at which the sum is smallest. The validation set (never used for gradient updates) is the tool that estimates this sum, and the test set gives the final, unbiased estimate once the choice has been made.

Task 3 – Maximum Likelihood Estimation: Bernoulli Model

QUESTION (4 Marks)

In one production batch, 50 components are inspected and 7 are found defective. Let each component be defective with unknown probability p, independently of the others.

a) Write the likelihood and log-likelihood function for the observed data.

b) Derive the maximum likelihood estimate of p and compute its value.

c) Using your estimate, calculate the probability that the next three inspected components are all acceptable, and comment on the reliability of the estimate for a small sample.

ANSWER

(a) Likelihood and log-likelihood

Let $x_i \in \{0,1\}$ (1 = defective), n = 50 and $k = \Sigma x_i = 7$. With independence:

$$L(p) = \prod p^{x_i}(1-p)^{1-x_i} = p^k(1-p)^{n-k} = p^7(1-p)^{43}$$

$$\ell(p) = \ln L(p) = 7 \ln p + 43 \ln(1-p)$$

(b) MLE

$$d\ell/dp = 7/p - 43/(1-p) = 0 \quad \Rightarrow \quad 7(1-p) = 43p \quad \Rightarrow \quad 7 = 50p$$

$$\hat{p} = k/n = 7/50 = 0.14$$

Second derivative: $d^2\ell/dp^2 = -7/p^2 - 43/(1-p)^2 < 0$, so this is a maximum.

(c) Next three acceptable and reliability

$$P(3 \text{ acceptable}) = (1 - 0.14)^3 = 0.86^3 \approx 0.636$$

Reliability: with only 50 samples the estimate is noisy. Its standard error is $\sqrt{(\hat{p}(1-\hat{p})/n)} = \sqrt{(0.14 \times 0.86/50)} \approx 0.049$, giving an approximate 95% interval of $0.14 \pm 0.096 \approx (0.04, 0.24)$. The true defect rate could therefore be much lower or higher, and the 0.636 figure inherits that uncertainty. MLE is consistent, so the estimate improves as n grows; with a small sample a Bayesian prior or larger batch is advisable.

Why the log is used. Taking the logarithm turns the product of 50 probabilities into a sum, which is easier to differentiate and numerically stable (the raw likelihood $0.14^7 \times 0.86^{43} \approx 1.6 \times 10^{-9}$ is already tiny), and because ln is strictly increasing, it has the same maximizer as L(p).

Intuition and check. The estimate $\hat{p} = k/n$ is simply the observed fraction of defectives, and the likelihood is largest exactly there: $\ell(0.14) = 7 \ln 0.14 + 43 \ln 0.86 \approx -13.76 - 6.49 = -20.25$, which is larger than $\ell(0.10) \approx -20.65$ and $\ell(0.20) \approx -20.86$. Expressed as an expected count, a batch of 1,000 components would be expected to contain about 140 defective parts if this estimate were exact.

Task 4 – MLE: Gaussian Model and Link to MSE

QUESTION (5 Marks)

A furnace temperature sensor records the readings 68, 70, 72, 74 and 76 °C, assumed to be drawn independently from a Gaussian distribution $N(\mu, \sigma^2)$.

a) Write the Gaussian likelihood and log-likelihood function.

b) Derive the MLE of μ and σ^2 and compute their values for the given readings.

c) Explain why the MLE of σ^2 is biased, compute the unbiased estimate using n − 1, and compare the two values.

d) For the regression model $y = wx + b + \varepsilon$ with $\varepsilon \sim N(0, \sigma^2)$, show that

maximizing the likelihood is equivalent to minimizing the mean squared error.

ANSWER

(a) Likelihood and log-likelihood

$$L(\mu,\sigma^2) = \prod (2\pi\sigma^2)^{\wedge}(-1/2)\ \exp[-(x_i-\mu)^2/(2\sigma^2)]$$

$$\ell(\mu,\sigma^2) = -(n/2)\ \ln(2\pi) - (n/2)\ \ln\ \sigma^2 - (1/(2\sigma^2))\ \Sigma(x_i-\mu)^2$$

(b) MLE of μ and σ²

$$\partial\ell/\partial\mu = (1/\sigma^2)\ \Sigma(x_i-\mu) = 0 \quad \Rightarrow \quad \hat{\mu} = (1/n)\ \Sigma x_i$$

$$\partial\ell/\partial\sigma^2 = -n/(2\sigma^2) + \Sigma(x_i-\mu)^2/(2\sigma^4) = 0 \quad \Rightarrow \quad \hat{\sigma}^2 = (1/n)\ \Sigma(x_i-\hat{\mu})^2$$

Numerically: $\hat{\mu} = (68+70+72+74+76)/5 = 360/5 =$ **72 °C**. Deviations: −4, −2, 0, 2, 4 → squares 16+4+0+4+16 = 40, so $\hat{\sigma}^2 = 40/5 =$ **8 °C²**.

(c) Bias and unbiased estimate

Because $\hat{\mu}$ is computed from the same data, the sample points lie closer to $\hat{\mu}$ than to the true μ, so $\Sigma(x_i-\hat{\mu})^2$ underestimates $\Sigma(x_i-\mu)^2$. Formally $E[\hat{\sigma}^2] = ((n-1)/n)\sigma^2$, i.e. the MLE is biased low (by factor (n−1)/n; one degree of freedom is used up estimating μ). The unbiased estimator is:

$$s^2 = (1/(n-1))\ \Sigma(x_i-\hat{\mu})^2 = 40/4 = 10\ °C^2$$

Comparison: 10 vs 8 – the unbiased value is 25% larger (ratio n/(n−1) = 5/4). The gap shrinks as n grows, so the bias matters mainly for small samples such as n = 5.

(d) MLE ⇔ MSE for y = wx + b + ε

Given $\varepsilon \sim N(0,\sigma^2)$, $y|x \sim N(wx+b, \sigma^2)$. For m samples:

$$\ell(w,b) = \Sigma\ \ln[(2\pi\sigma^2)^{\wedge}(-1/2)\ \exp(-(y_i-wx_i-b)^2/(2\sigma^2))] = -(m/2)\ \ln(2\pi\sigma^2) - (1/(2\sigma^2))\ \Sigma(y_i-wx_i-b)^2$$

The first term does not depend on w, b and $\sigma^2 > 0$ is a constant factor, so maximizing ℓ over (w,b) is the same as minimizing $\Sigma(y_i-wx_i-b)^2$, i.e. the mean squared error $(1/m)\Sigma(y_i-\hat{y}_i)^2$. Hence least-squares regression is maximum-likelihood estimation under Gaussian noise.

Why the MLE is biased – derivation. Write $\Sigma(x_i-\hat{\mu})^2 = \Sigma(x_i-\mu)^2 - n(\hat{\mu}-\mu)^2$. Taking expectations, $E[\Sigma(x_i-\mu)^2] = n\sigma^2$ and $E[n(\hat{\mu}-\mu)^2] = n\cdot(\sigma^2/n) = \sigma^2$, so $E[\Sigma(x_i-\hat{\mu})^2] = (n-1)\sigma^2$. Hence $E[\hat{\sigma}^2] = ((n-1)/n)\sigma^2 < \sigma^2$, while $E[s^2] = \sigma^2$. For the furnace data the standard deviations are $\hat{\sigma} = \sqrt{8} \approx 2.83$ °C and $s = \sqrt{10} \approx 3.16$ °C. Since the sample is only 5 readings, using s (or $s^2 = 10$) is the more honest description of the sensor

spread.

Task 5 – Building a Machine Learning Algorithm

QUESTION (5 Marks)

The factory wants to predict tool wear y (in units of 0.1 mm) from running time x (in hours). Four observations are available: (x, y) = (1, 3), (2, 5), (3, 6), (4, 9). The proposed model is $\hat{y} = wx + b$ with cost $J = (1/m)\ \Sigma\ (\hat{y} - y)^2$.

a) Identify the dataset, model, cost function and optimization procedure that together form this learning algorithm.

b) Derive the expressions for $\partial J/\partial w$ and $\partial J/\partial b$.

c) Starting from w = 0 and b = 0 with learning rate $\alpha = 0.05$, compute the gradients and perform one gradient descent update. Report the new w and b.

d) Differentiate between parameters and hyperparameters in this problem, and explain the effect of choosing α too large or too small.

ANSWER

(a) Four components

Component	In this problem
Dataset	(x,y) = (1,3), (2,5), (3,6), (4,9); m = 4 (running hours → tool wear)
Model	$\hat{y} = wx + b$ (linear regression)
Cost function	$J(w,b) = (1/m)\ \Sigma(\hat{y}_i - y_i)^2$ (mean squared error)
Optimization	Gradient descent: $w \leftarrow w - \alpha\ \partial J/\partial w$, $b \leftarrow b - \alpha\ \partial J/\partial b$

(b) Gradients

$$\partial J/\partial w = (2/m)\ \Sigma\ (wx_i + b - y_i)\ x_i$$

$$\partial J/\partial b = (2/m)\ \Sigma\ (wx_i + b - y_i)$$

(Obtained by the chain rule: $d/dw\ (\hat{y}-y)^2 = 2(\hat{y}-y)\cdot\partial\hat{y}/\partial w$, with $\partial\hat{y}/\partial w = x$ and $\partial\hat{y}/\partial b = 1$.)

(c) One gradient-descent step (w = 0, b = 0, α = 0.05)

Errors $\hat{y} - y = -3, -5, -6, -9$ (all predictions are 0).

$$\partial J/\partial w = (2/4)[(-3)(1)+(-5)(2)+(-6)(3)+(-9)(4)] = 0.5(-3-10-18-36) = 0.5(-67) = -33.5$$

$\partial J/\partial b = (2/4)(-3-5-6-9) = 0.5(-23) = -11.5$

$w_1 = 0 - 0.05(-33.5) = 1.675$

$b_1 = 0 - 0.05(-11.5) = 0.575$

Check: J falls from 37.75 (at w=b=0) to ≈ 1.21 after this step.

Check of the update. Before the step J(0,0) = (9+25+36+81)/4 = 37.75. After the step the predictions are $\hat{y} = 1.675x + 0.575 = 2.25, 3.925, 5.6, 7.275$, with errors −0.75, −1.075, −0.4, −1.725, so J = (0.5625+1.1556+0.16+2.9756)/4 ≈ 1.213. A single step has reduced the cost by about 97%, because the initial point was far from the minimum. For reference, the exact least-squares solution (normal equations) is w = Sxy/Sxx = 9.5/5 = 1.9 and $b = \bar{y} - w\,\bar{x} = 5.75 - 1.9(2.5) = 1.0$, so repeated gradient-descent updates would converge towards (w, b) = (1.9, 1.0).

(d) Parameters vs hyperparameters

- **Parameters** (w, b) are learned from the data by gradient descent.
- **Hyperparameters** (α, number of iterations, batch size, model form) are set before training and tuned on validation data.
- **α too large:** updates overshoot the minimum; loss oscillates or diverges. **α too small:** convergence is very slow and may stall in flat regions, wasting computation.

Why the gradient step moves in this direction. Both gradients are negative, meaning that increasing w or b would decrease the cost; the update $w \leftarrow w - \alpha\partial J/\partial w$ therefore moves the parameters upward. The step size is α times the gradient, so a steeper slope produces a bigger move. With α = 0.05 the step on w is 1.675 and on b is 0.575, which is stable here. If α were 0.5 the first step would give w = 16.75, b = 5.75 and the cost would explode, illustrating the "too large" case.

Task 6 – Multilayer Perceptron: Architecture and Forward Pass

QUESTION (4 Marks)

A Multilayer Perceptron has 2 inputs, one hidden layer with 2 ReLU neurons, and 1 sigmoid output neuron. The input is x = (0.5, 1.0). Hidden weights and biases: h_1 uses weights (0.4, 0.2) and bias 0.1; h_2 uses weights (−0.6, 0.5) and bias −0.2. Output weights are $v_1 = 0.8$ and $v_2 = 0.3$ with output bias −0.1.

a) Draw the network and count the number of trainable parameters.

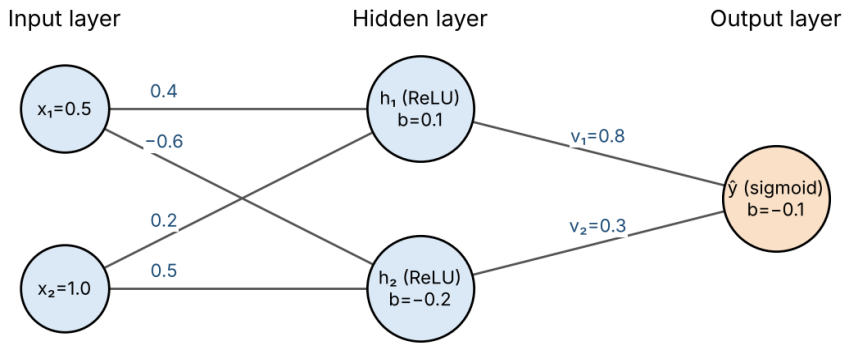
b) Perform the forward pass and compute h_1, h_2 and the output $\hat{y}$.

c) Explain why a hidden layer with a non-linear activation is necessary, and

what happens if all activations are linear.

ANSWER

(a) Network and parameter count



Parameters: hidden layer 2×2 weights + 2 biases = 6; output layer 2 weights + 1 bias = 3. **Total = 9 trainable parameters.**

Layer	Weights	Biases	Parameters
Input → hidden (2×2)	4	2	6
Hidden → output (2×1)	2	1	3
Total	6	3	9

(b) Forward pass, x = (0.5, 1.0)

$z_1 = 0.4(0.5) + 0.2(1.0) + 0.1 = 0.5 \Rightarrow h_1 = \text{ReLU}(0.5) = 0.5$

$z_2 = -0.6(0.5) + 0.5(1.0) - 0.2 = 0.0 \Rightarrow h_2 = \text{ReLU}(0) = 0$

$z_o = 0.8(0.5) + 0.3(0) - 0.1 = 0.3$

$\hat{y} = \sigma(0.3) = 1/(1+e^{-0.3}) \approx 0.5744$

(c) Why non-linear hidden activation

Without non-linearity, each layer computes an affine map, and a composition of affine maps is again affine: $W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2)$. If all activations are linear, the network collapses to a single linear model, however many layers it has, and cannot represent non-linear decision boundaries (e.g. XOR). A non-linear activation such as ReLU lets the hidden layer build piece-wise non-linear features, giving the network its extra representational power (universal approximation).

Interpretation. The output $\hat{y} = 0.5744$ is a probability: the network thinks there is about a 57% chance that the component is defective. With a 0.5 threshold the component would be classified as defective, but the network is not confident. Hidden unit h_2 is exactly at the ReLU kink ($z_2 = 0$), so it is inactive and passes no signal to the output. The number of operations in this forward pass is small – 6 multiplications and additions in the hidden layer and 2 in the output layer – which is why the same procedure scales to large networks.

Task 7 – Back-propagation Algorithm

QUESTION (5 Marks)

Use the network and input of Task 6 with target $t = 1$ and loss $L = \frac{1}{2}(t - \hat{y})^2$. The output neuron is sigmoid and the hidden neurons are ReLU.

a) Write the chain-rule expressions for $\partial L/\partial v_1$ and $\partial L/\partial w_{11}$, where w_{11} is the weight from the first input to h_1.

b) Compute $\partial L/\partial v_1$, $\partial L/\partial v_2$, $\partial L/\partial$(output bias) and $\partial L/\partial w_{11}$ numerically.

c) Update v_1 and w_{11} using learning rate $\eta = 0.1$.

d) Explain why back-propagation is more efficient than computing every derivative independently.

ANSWER

(a) Chain-rule expressions

Let $z_o = v_1 h_1 + v_2 h_2 + c$ (c = output bias), $\hat{y} = \sigma(z_o)$, $L = \frac{1}{2}(t-\hat{y})^2$, $z_1 = w_{11} x_1 + w_{12} x_2 + b_1$, $h_1 = \mathrm{ReLU}(z_1)$.

$$\partial L/\partial v_1 = (\partial L/\partial \hat{y})(\partial \hat{y}/\partial z_o)(\partial z_o/\partial v_1) = (\hat{y}-t) \cdot \hat{y}(1-\hat{y}) \cdot h_1$$

$$\partial L/\partial w_{11} = (\hat{y}-t) \cdot \hat{y}(1-\hat{y}) \cdot v_1 \cdot \mathrm{ReLU}'(z_1) \cdot x_1$$

(b) Numerical gradients ($\hat{y} = 0.5744$, $t = 1$)

$$\hat{y}(1-\hat{y}) = 0.5744 \times 0.4256 = 0.2445; \quad \hat{y} - t = -0.4256$$

$$\delta_o = \partial L/\partial z_o = (-0.4256)(0.2445) = -0.1040$$

$$\partial L/\partial v_1 = \delta_o\, h_1 = -0.1040 \times 0.5 = -0.0520$$

$$\partial L/\partial v_2 = \delta_o\, h_2 = -0.1040 \times 0 = 0$$

$$\partial L/\partial c = \delta_o = -0.1040$$

$$\partial L/\partial h_1 = \delta_o\, v_1 = -0.1040 \times 0.8 = -0.0832; \quad \mathrm{ReLU}'(0.5) = 1$$

$$\partial L/\partial w_{11} = -0.0832 \times 1 \times x_1 = -0.0832 \times 0.5 = -0.0416$$

(Loss before the update: $L = \tfrac{1}{2}(0.4256)^2 \approx 0.0905$. Note $h_2 = 0$ gives zero gradient for v_2, and ReLU′(0) is taken as 0 by convention.)

(c) Updates with $\eta = 0.1$

$$v_1 \leftarrow 0.8 - 0.1(-0.0520) = 0.8052$$

$$w_{1,1} \leftarrow 0.4 - 0.1(-0.0416) = 0.4042$$

Both weights increase, pushing $\hat{y}$ closer to the target 1.

Quantity	Expression	Value
Output error signal δ_o	$(\hat{y} - t)\cdot\hat{y}(1 - \hat{y})$	−0.1040
$\partial L/\partial v_1$	$\delta_o \cdot h_1$	−0.0520
$\partial L/\partial v_2$	$\delta_o \cdot h_2$	0
$\partial L/\partial c$ (output bias)	δ_o	−0.1040
$\partial L/\partial h_1$	$\delta_o \cdot v_1$	−0.0832
$\partial L/\partial z_1$	$\partial L/\partial h_1 \cdot \mathrm{ReLU}'(z_1)$	−0.0832
$\partial L/\partial w_{1,1}$	$\partial L/\partial z_1 \cdot x_1$	−0.0416

Check of the update. After the update, $z_1 = 0.4042(0.5) + 0.2(1.0) + 0.1 = 0.5021$, $h_1 = 0.5021$, $z_o = 0.8052(0.5021) + 0.3(0) - 0.1 = 0.3043$, so $\hat{y} = \sigma(0.3043) \approx 0.5755$. The loss falls from $L = 0.0905$ to $L \approx 0.0901$, a small but correct decrease in the direction of the target; repeated updates would continue this trend. The step is small because the sigmoid slope $\hat{y}(1-\hat{y}) \approx 0.24$ is multiplied into every gradient and $\eta = 0.1$ is modest.

(d) Why back-propagation is efficient

Back-propagation reuses intermediate results: the error signal δ at a layer is computed once and passed backwards, so each layer's gradients reuse those of the layer above (dynamic programming over the computational graph). The full gradient of all P parameters costs about the same as one forward pass (O(P)). Computing each derivative independently (or by finite differences) would repeat the shared chain-rule products for every parameter, requiring roughly P separate forward/backward evaluations ($O(P^2)$) – infeasible for networks with millions of weights.

Example. For a network with 10 layers, the backward signal reaching layer 1 is multiplied by 10 weights and 10 derivatives. Even if the weights are around 1, the activation factors alone limit it to 9.54×10^{-7}. Back-propagation also needs this product to be computed for every one of the earlier layers, so the layer nearest the output may be

learning 10^6 times faster than the layer nearest the input.

Task 8 – Vanishing Gradient and Activation Functions

QUESTION (3 Marks)

The factory deepens its network to 10 hidden layers, all using sigmoid activation. Training loss barely decreases in the early layers.

a) Show that the derivative of the sigmoid is at most 0.25 and calculate the upper bound 0.25^{10} on the product of activation derivatives across 10 layers.

b) Explain how this causes vanishing gradients and what it does to learning in the early layers.

c) Suggest two remedies and justify each.

ANSWER

(a) Bound on the sigmoid derivative

$\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Let $s = \sigma(z) \in (0,1)$: $f(s) = s(1-s)$.

$f'(s) = 1 - 2s = 0 \Rightarrow s = ½$ (at $z = 0$), $f(½) = ¼ \Rightarrow \sigma'(z) \leq 0.25$

Across 10 layers: $(0.25)^{10} = 1/4^{10} = 1/1{,}048{,}576 \approx 9.54 \times 10^{-7}$

Depth n	Upper bound 0.25^n
1	0.25
2	0.0625
5	9.77×10^{-4}
10	9.54×10^{-7}

(b) Effect

By the chain rule the gradient reaching layer 1 contains a product of ten activation derivatives (times weights). With each factor ≤ 0.25 (and typically much smaller when units saturate) the gradient shrinks exponentially with depth – below 10^{-6} of its original size even in the best case. Early layers therefore receive almost zero updates ($w \leftarrow w - \eta \cdot \approx 0$), learn extremely slowly and the loss barely decreases, while the late layers train normally.

(c) Remedies

- **ReLU / Leaky ReLU activations:** derivative is 1 for positive inputs, so gradients

are not repeatedly shrunk by the activation.

- **Better initialization and normalization (Xavier/He init, batch normalization) or residual (skip) connections:** keep activations and gradient magnitudes near 1 across layers, and skip connections give gradients a direct path to early layers.

Additional remark. Besides saturating derivatives, the opposite problem – exploding gradients – occurs when weights are large. Gradient clipping is a standard fix for that case. For the factory model, a sensible practical combination is ReLU activations, He initialization, batch normalization and Adam, which together keep the loss decreasing in all layers.

Task 9 – Batch, Stochastic and Mini-batch Gradient Descent

QUESTION (3 Marks)

The factory trains on N = 1,000,000 inspection records and compares batch gradient descent, stochastic gradient descent (SGD) and mini-batch gradient descent with a batch size of 100.

a) Calculate the number of parameter updates in one epoch for each method.

b) Compare the methods in terms of gradient noise, memory use, cost per update and convergence behaviour.

c) Recommend one method for this dataset and justify the choice.

ANSWER

(a) Updates per epoch (N = 1,000,000)

Method	Batch size	Updates per epoch
Batch GD	1,000,000	1
SGD	1	1,000,000
Mini-batch	100	1,000,000 / 100 = 10,000

(b) Comparison

Aspect	Batch	SGD	Mini-batch (100)
Gradient noise	None (exact gradient)	Very high	Moderate (variance $\propto$ 1/100)
Memory	Whole dataset per step – highest	Lowest (1 sample)	Low (100 samples)

Aspect	Batch	SGD	Mini-batch (100)
Cost per update	Very high (1M samples)	Very cheap but poor hardware use	Cheap; vectorized on GPU
Convergence	Smooth but only 1 step/epoch; slow wall-clock	Fast early progress, oscillates around minimum	Fast and fairly stable; noise helps escape poor minima

Quantitative comparison. The variance of a mini-batch gradient estimate is σ^2/B, where σ^2 is the single-sample gradient variance. SGD ($B = 1$) has the largest variance σ^2, batch GD has zero, and a batch of 100 reduces the noise standard deviation by a factor of $\sqrt{100} = 10$ compared with SGD. If 1,000,000 samples of 4 float32 features (16 bytes each) are held in memory the data alone is 16 MB, but at 500 features per sample it becomes 2 GB, which shows why computing a full-batch gradient can become impractical; a mini-batch of 100 samples needs only about 200 KB. Because GPUs execute matrix operations in parallel, a batch of 100 costs hardly more wall-clock time than a single sample, so the mini-batch method makes much better use of the hardware per update.

(c) Recommendation

Use **mini-batch gradient descent** (e.g. batch size 100–256). It gives 10,000 updates per epoch instead of 1, so learning is far faster than batch GD, its gradient is far less noisy than single-sample SGD, and it fits in memory and exploits GPU matrix parallelism. Its mild noise also aids generalization.

Practical choice of batch size. Typical batch sizes are powers of two (32, 64, 128, 256) because of memory alignment on GPUs. Larger batches give smoother gradients but fewer updates per epoch and can generalize slightly worse; smaller batches are noisier but update more often. A batch size of 100 is a reasonable compromise for this dataset and can be tuned on the validation set together with the learning rate.

Task 10 – SGD Variants: Momentum, RMSProp and Adam

QUESTION (3 Marks)

A single weight starts at $w_0 = 2$. Two successive gradients are $g_1 = 4$ and $g_2 = 1$, with learning rate $\eta = 0.1$. For momentum, use $v_t = \beta v_{t-1} + g_t$ and $w_t = w_{t-1} - \eta v_t$ with $\beta = 0.9$ and $v_0 = 0$.

a) Compute w_1 and w_2 using plain SGD and then using SGD with momentum.

b) Explain which problem of plain SGD is addressed by momentum, by RMSProp and by Adam.

c) State one situation where Adam is preferred over plain SGD and one situation where plain SGD may still be preferred.

ANSWER

(a) Updates ($w_0 = 2$, $\eta = 0.1$, $g_1 = 4$, $g_2 = 1$)

Plain SGD: $w_t = w_{t-1} - \eta g_t$

$$w_1 = 2 - 0.1(4) = 1.6$$

$$w_2 = 1.6 - 0.1(1) = 1.5$$

Momentum ($\beta = 0.9$, $v_0 = 0$):

$$v_1 = 0.9(0) + 4 = 4 \quad \Rightarrow \quad w_1 = 2 - 0.1(4) = 1.6$$

$$v_2 = 0.9(4) + 1 = 4.6 \quad \Rightarrow \quad w_2 = 1.6 - 0.1(4.6) = 1.14$$

Momentum moves further (1.14 vs 1.5) because past gradients accumulate in v, so the earlier large gradient keeps pushing the weight.

Comparison of the two trajectories: SGD moves −0.4 and then −0.1 (total −0.5); momentum moves −0.4 and then −0.46 (total −0.86). The second momentum step is larger than the gradient alone would justify, because the velocity $v_2 = 4.6$ carries 90% of the previous gradient. This is exactly the "rolling ball" effect: when successive gradients point in a similar direction, the steps accumulate and training accelerates; when gradients alternate in sign, they cancel and oscillation is damped.

Update rules. RMSProp: $s_t = \rho s_{t-1} + (1-\rho) g_t^2$, $w_t = w_{t-1} - \eta\, g_t/(\sqrt{s_t} + \varepsilon)$. Adam: $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$, $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$, bias-corrected $\hat{m}_t = m_t/(1-\beta_1^t)$, $\hat{v}_t = v_t/(1-\beta_2^t)$, $w_t = w_{t-1} - \eta\, \hat{m}_t/(\sqrt{\hat{v}_t} + \varepsilon)$, with typical $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

(b) Problem addressed by each

- **Momentum:** oscillation across steep directions and slow progress along shallow valleys/plateaus – accumulating a velocity damps oscillations and speeds consistent directions.
- **RMSProp:** a single global learning rate suits no parameter well when gradient scales differ; it divides by a running RMS of gradients, giving per-parameter adaptive step sizes (and tames exploding/vanishing step sizes).
- **Adam:** combines both – momentum (first moment) plus RMSProp-style scaling

(second moment) with bias correction – for fast, robust convergence with little tuning.

(c) When to choose

- **Adam preferred:** sparse or badly-scaled gradients, noisy objectives, or deep/complex networks where quick convergence with default hyperparameters is wanted.
- **Plain SGD (with momentum/decay) may be preferred:** when best final generalization is needed on well-conditioned problems (e.g. large-scale image models) or when memory is tight, since Adam stores two extra vectors per parameter.

Summary. Momentum fixes oscillation and slow movement in flat valleys, RMSProp fixes the problem that one learning rate does not suit parameters with very different gradient scales, and Adam combines the two so it is a robust default. Plain SGD has the fewest hyperparameters and the lowest memory use, and with a well-tuned learning-rate schedule it often generalizes as well as adaptive methods.

Task 11 – Curse of Dimensionality

QUESTION (3 Marks)

The factory plans to extend the sensor set from 4 features to 100 features. Every feature is scaled to [0, 1] and divided into 10 equal bins. Only 5,000 labelled records are available.

a) Calculate the number of cells in the feature space for d = 4 and d = 100, and compare it with 5,000 records.

b) For uniformly distributed data, a sub-cube must contain 10% of the data. Using r = 0.1^(1/d), compute the required edge length for d = 2, 10 and 100 and interpret the result.

c) Explain the consequences for a k-nearest neighbour classifier and suggest two ways to reduce the problem.

ANSWER

(a) Number of cells (10 bins per feature)

d = 4: 10^4 = 10,000 cells; d = 100: 10^{100} cells

With 5,000 records there are only 0.5 records per cell on average for d = 4 (already sparse, over half the cells are empty), and $5{,}000/10^{100} = 5\times10^{-97}$ per cell for d = 100

– virtually every cell is empty, so the data cover a negligible fraction of the space.

(b) Edge length for a sub-cube containing 10% of the data: r = 0.1^(1/d)

d	r = 0.1^(1/d)	Interpretation
2	0.316	Covers about 32% of each axis
10	0.794	Covers about 79% of each axis
100	0.977	Covers about 98% of each axis

To capture just 10% of the data in high dimensions, the "local" neighbourhood must span nearly the entire range of every feature. Neighbourhoods are no longer local, and data points become almost equally far from each other.

Fraction of space covered. The result in part (a) means that to have just 5 data points per cell in 4 dimensions one would need 50,000 records, and in 100 dimensions the number is astronomically larger (5×10^{100}). The general rule is that the data needed for a given coverage grows exponentially with d. The edge-length result in (b) can also be read as follows: a hypercube with edge 0.977 in 100 dimensions has volume $0.977^{100} \approx 0.1$, so even though it spans 97.7% of every axis it contains only 10% of the points – almost all the volume lies near the boundary of the cube, far from its centre.

(c) Consequences for k-NN and mitigation

k-NN relies on nearby points being similar. In 100 dimensions, distances concentrate (nearest and farthest neighbours have almost the same distance), so "nearest" neighbours are not informative; the classifier becomes inaccurate, needs exponentially more data, and each query is expensive (distance over 100 features).

- **Dimensionality reduction/feature selection** (PCA, autoencoders, removing irrelevant or correlated sensors).
- **More data or regularization / simpler models** (collect more records, use domain-driven features, metric learning) to reduce sparsity.

Distance concentration. In high dimensions the ratio of the largest to the smallest pairwise distance tends to 1, so the idea of a "nearest" neighbour loses meaning. Irrelevant or noisy sensor channels add to the distance and swamp the informative ones. Reducing d with PCA keeps the directions of largest variance, and feature selection removes uninformative channels; both make the distances meaningful again and cut k-NN's memory and query cost. Using a parametric model (for example a regularized neural network) instead of a local method is another way to avoid relying on

neighbourhoods.

Task 12 – Design Challenge: Selecting and Adapting a Learning Strategy

QUESTION (8 Marks)

Changed Requirement: The factory initially analyses 5,000 records with 4 sensor features using a logistic regression model trained by full-batch gradient descent. It must now process 2,000,000 records with 500 sensor features per component, predict the defect probability in near real time, and handle correlated features. Decide whether to retain, modify or redesign the strategy.

A. Impact Analysis (2 Marks)

a) Explain how the larger dataset and 500 features affect computation, memory and the training procedure.

b) Discuss why dimensionality, feature correlation and gradient behaviour become important.

B. Evaluation of the Original Approach (2 Marks)

a) Evaluate whether a linear model trained with full-batch gradient descent is sufficient for the new requirement.

b) Explain the limitations of the original approach in terms of capacity, optimization cost and the curse of dimensionality.

C. Adaptation or Redesign (2 Marks)

a) Propose a redesigned strategy using a Multilayer Perceptron trained with back-propagation and a suitable SGD variant.

b) Explain how maximum likelihood (the loss function), regularization, dimensionality reduction and the learning rate can be incorporated.

D. Evidence-Based Defence (2 Marks)

a) Propose experiments and evaluation measures to compare the redesigned approach with the original one.

b) Justify the final design using mathematical reasoning, generalization, computational needs and data characteristics.

ANSWER

Decision: redesign – retain the probabilistic (MLE/cross-entropy) framework of logistic regression, but replace the linear model by an MLP and full-batch GD by mini-batch training with an adaptive optimizer.

A. Impact Analysis

(a) Computation, memory, training. The data matrix grows from 5,000×4 = 2×10^4 values to 2,000,000×500 = 10^9 values (≈ 4 GB in float32, 50,000× more). One full-batch gradient costs O(mD) = 10^9 operations, and full-batch GD would give only one update per pass over all the data, requiring the whole matrix in memory. Training must therefore stream mini-batches, use vectorized/GPU computation, and shuffle data every epoch. Inference must stay at O(D) per sample to satisfy near-real-time prediction.

(b) Dimensionality, correlation, gradient behaviour. With 500 features the space is far sparser (curse of dimensionality) and the risk of overfitting grows with the parameter count. Correlated features make the loss surface ill-conditioned (long narrow valleys: the Hessian has a large condition number), so a single learning rate causes zig-zag and slow convergence, and weight estimates become unstable. Differing feature scales and deeper networks also affect gradient magnitude (vanishing/exploding), so normalization and adaptive optimizers matter.

Quantitative illustration. Parameter count: a logistic model has D + 1 = 501 parameters, which is small relative to 2,000,000 records – so variance is actually low – but the model is restricted to a linear boundary, so bias is the main risk. A full-batch update costs about $2\times m\times D = 2\times2\times10^6\times500 = 2\times10^9$ floating-point operations; with a mini-batch of 256 it costs 2.6×10^5 operations, about 7,800 times cheaper, while using an unbiased estimate of the same gradient. If 1,000 epochs of full-batch descent were needed, only 1,000 updates would occur, compared with 7.8 million updates by mini-batch training in the same number of epochs.

Effect of correlated features. If two sensors are highly correlated (for example temperature of adjacent furnace zones), the loss is almost flat along the direction $w_1 - w_2$ and steep in the direction $w_1 + w_2$. The ratio of curvatures (the condition number κ) can be very large, and gradient descent needs on the order of κ iterations to converge. Decorrelating the features with PCA or whitening makes the loss surface nearly circular, $\kappa \to 1$, and gradient methods converge quickly.

B. Evaluation of the Original Approach

(a) Sufficiency. A linear logistic model with full-batch GD is not sufficient. It is only able to form a linear decision boundary, and full-batch GD is too slow and memory-hungry at 2M × 500.

(b) Limitations. Capacity: the linear boundary cannot capture non-linear interactions among 500 sensors (high bias / underfit). **Optimization cost:** each update needs a full

pass (10^9 operations); it converges slowly on correlated, ill-conditioned features. **Curse of dimensionality:** with 500 unregularized weights and correlated inputs the estimate is unstable and noisy, and local methods are unreliable in such a sparse space.

Overall assessment. Because the new requirements change the scale (400× more records, 125× more features), the time constraint (near real time) and the statistical character of the data (strong correlation), none of the three components of the original learning algorithm – model, loss/optimizer procedure and data handling – remains adequate without change. The model class (linear), the optimizer (full batch) and the data pipeline (in memory) all need modification, so the right decision is to **redesign**, while keeping the part that still works: the probabilistic MLE formulation with a sigmoid output.

C. Adaptation or Redesign

(a) Proposed design. Standardize inputs → MLP 500 → 128 → 64 → 1 with ReLU hidden units (He initialization) and sigmoid output for the defect probability; train with back-propagation using mini-batches (e.g. 256) and Adam (or SGD with momentum). Inference is a small forward pass ($\approx 500 \cdot 128 + 128 \cdot 64 + 64 \approx 72$k multiply-adds), suitable for real time.

(b) Incorporating the required elements.

- **Maximum likelihood loss:** treat $y|x \sim \text{Bernoulli}(\hat{y})$; maximizing likelihood = minimizing binary cross-entropy $J = -(1/m)\Sigma[y \ln \hat{y} + (1-y) \ln(1-\hat{y})]$ (class weights or focal loss for the rare defect class).
- **Regularization:** L2 weight decay $\lambda\|W\|^2$ (a Gaussian prior, i.e. MAP), dropout and early stopping on validation loss.
- **Dimensionality reduction:** PCA (or an autoencoder bottleneck) to decorrelate the 500 features and keep, e.g., 95% variance; or feature selection by correlation/mutual information.
- **Learning rate:** Adam with $\eta \approx 10^{-3}$, decayed by schedule/plateau detection and tuned on validation data; warm-up if unstable.

Component	Original strategy	Redesigned strategy
Model	Logistic regression (linear, 501 parameters)	MLP 500→128→64→1, ReLU + sigmoid (≈ 72,000 parameters)
Loss	Binary cross-entropy (MLE)	Binary cross-entropy (MLE) + L2 penalty, class weights
Optimizer	Full-batch gradient descent	Mini-batch (256) Adam with learning-rate decay

Component	Original strategy	Redesigned strategy
Inputs	4 raw features	500 standardized features, PCA to ~100–150 components
Regularization	None	Weight decay, dropout (0.2–0.5), early stopping
Per-sample inference	≈ 500 operations	≈ 72,000 multiply-adds (still well under a millisecond)

Training procedure. 1) Standardize each feature using the training mean and variance (and apply the same values at inference). 2) Fit PCA on the training set only. 3) Initialize weights with He initialization. 4) For each epoch: shuffle, loop over mini-batches, run the forward pass, compute the cross-entropy plus weight-decay loss, back-propagate and update with Adam. 5) Evaluate on the validation set after each epoch, reduce the learning rate when validation loss plateaus, and stop early when it has not improved for several epochs. 6) Report final results once on the untouched test set.

D. Evidence-Based Defence

(a) Experiments and measures. Split the 2M records into train/validation/test (stratified, time-ordered to avoid leakage). Compare (1) baseline logistic regression + full-batch GD, (2) logistic regression + mini-batch SGD, (3) the redesigned MLP (with/without PCA and regularization), over several random seeds. Measure:

- Predictive quality: precision, recall, F1, PR-AUC/ROC-AUC, plus log-loss and calibration, since defects are rare.
- Generalization: training vs. validation curves and the generalization gap; k-fold validation.
- Efficiency: time per epoch, time to reach target loss, memory, and inference latency per component (must meet the real-time limit).
- Ablations: effect of optimizer, batch size, learning rate, weight decay and PCA dimension.

(b) Justification. Mathematically, cross-entropy is the MLE criterion for Bernoulli labels, giving well-calibrated probabilities; the MLP adds the capacity the linear model lacks, while L2/dropout/early stopping and PCA control the variance and the dimensionality problem, which keeps the gap small. Mini-batch updates give $N/B \approx 7{,}800$ updates per epoch (for $B = 256$) instead of one, and Adam's per-parameter scaling handles the ill-conditioning caused by correlated features. Computationally, the batches fit in memory

and use GPU parallelism, and inference is a cheap forward pass. With 2M records the data are rich enough to support the MLP's capacity. If experiments show that the MLP gives no significant gain in recall/PR-AUC over the baseline, the simpler regularized logistic model should be retained – the decision is evidence-driven.

Final decision. Retain only the maximum-likelihood principle and the sigmoid probability output of the original system; modify the loss with regularization; and redesign the model, optimizer and data pipeline. The design is accepted if, on the held-out test set, it improves recall at a fixed precision (or PR-AUC) over the baseline by a statistically significant margin, keeps the generalization gap small, and meets the real-time latency target. If it does not, the simpler regularized logistic model with mini-batch SGD and PCA is the fallback, because the evidence – not the complexity of the model – decides the final strategy.

Summary of Key Results

Task	Result
3	$\hat{p} = 0.14$; P(3 acceptable) ≈ 0.636
4	$\hat{\mu} = 72$, $\hat{\sigma}^2 = 8$, unbiased $s^2 = 10$
5	$\partial J/\partial w = -33.5$, $\partial J/\partial b = -11.5$; $w = 1.675$, $b = 0.575$
6	$h_1 = 0.5$, $h_2 = 0$, $\hat{y} \approx 0.5744$; 9 parameters
7	$\partial L/\partial v_1 = -0.0520$, $\partial L/\partial v_2 = 0$, $\partial L/\partial c = -0.1040$, $\partial L/\partial w_{11} = -0.0416$; $v_1 = 0.8052$, $w_{11} = 0.4042$
8	$0.25^{10} \approx 9.54 \times 10^{-7}$
9	1 / 1,000,000 / 10,000 updates per epoch
10	SGD: 1.6, 1.5; momentum: 1.6, 1.14
11	10^4 and 10^{100} cells; r = 0.316, 0.794, 0.977